

Do not use the STL <list> or Templates in your code. Do not change the specification or member functions interfaces or add extra member variables, if you do, you will get zero marks. Also do not use structs.

1. Go back to your `ListOfContacts` and related classes from Lab 5b and add an overloaded constructor to the **Contact** class. It has the signature below:
 - `Contact (string n, int num);`
2. Also add an overloaded constructor to the **NodeOfContact** class. It has the signature below:
 - `NodeOfContact(string n, int num);`
3. And then add the following member functions to the **ListOfContacts** class:
 - `bool insertBack(string, int);`
 - `bool deleteBack();`
 - `Contact deleteContact(string nameToDelete);`
 - `bool findContact(string nameToFind);`
 - `bool isEmpty() // return true if list empty`
 - a copy constructor for the `ListOfContacts`
 - an overloaded copy assignment operator for the `ListOfContacts` // You must use the Copy and Swap idiom see note 2 below.
 - `ostream& displayList(ostream&) const`
 - Then add an overloaded stream insertion operator `<<` to work with `ListOfContacts` objects, it has the signature below:
`friend ostream& operator<<(ostream& outputStream, const ListOfContacts&);`
4. Implement a main function (in a separate source.cpp file) to test your new `ListofContacts` class functionality.

Note 1: You can use the C++ string class, include `<string>` and to compare two Strings see the examples in the link here: <https://www.techiedelight.com/compare-two-strings-in-cpp/>

Note 2: The Copy and Swap idiom is explained here and in the lecture notes <https://stackoverflow.com/questions/3279543/what-is-the-copy-and-swap-idiom>

And there is a Copy and Swap idiom copy assignment List example below:
<https://stackoverflow.com/questions/30857668/assignment-operator-in-linked-list-c>